Informe TP4 Integración con JavaScript

Integrantes: Mateo Aybar, Ignacio Bazán, Ivo Sampaolesi y Alex Taborda.

En esta entrega seguimos trabajando sobre el sitio de cba.urban e incorporamos JavaScript para darle funcionamiento al catálogo, aplicando así conceptos que efectivamente, vimos en clase, al detalle de los productos y al carrito. Partimos de las páginas que ya teníamos realizadas en HTML y CSS, manteniendo el diseño general del proyecto.

El principal cambio fue cargar los productos desde un archivo JSON mediante `fetch()`. Esto nos permitió generar el catálogo, aplicar filtros y mostrar el detalle de cada prenda usando los datos del mismo archivo. También incorporamos el almacenamiento del carrito en el navegador y el funcionamiento de las páginas de contacto, perfil y confirmación.

Páginas trabajadas

Inicio, Ofertas y Perfil: incorporamos el contador del carrito en el encabezado y los enlaces a la página de contacto. En Inicio y Ofertas, el contador muestra la cantidad de unidades guardadas. En Perfil se pueden editar los datos y guardarlos en el navegador. El contenido principal de estas secciones se mantiene definido en HTML.

Catálogo (`catalogo.html`): los productos se cargan desde un archivo JSON y se muestran mediante JavaScript. Cada producto presenta su imagen, nombre, precio, un enlace para ver el detalle y un botón para agregarlo al carrito. También funcionan la búsqueda por texto y los filtros por categoría y talle, que se pueden combinar.

Detalle del producto (`producto.html`): la página muestra la información de la prenda seleccionada en el catálogo. Incluye imagen, nombre, precio, descripción, talles disponibles y un formulario para elegir color, talle y cantidad.

Carrito (`carrito.html`): muestra los productos guardados con su talle, color y cantidad. Las variantes de una misma prenda se identifican por separado. También permite eliminar una variante, agregar prendas desde la sección “Te puede interesar” y consultar el subtotal, el envío estimado y el total.

Confirmación de compra (`confirmacion.html`): muestra el resumen del pedido, sus productos y los importes. Si se abre la página sin un pedido pendiente, informa que no hay una compra para confirmar y no vacía el carrito.

Contacto y confirmación de contacto: incorporamos un formulario de consultas y una página que muestra el mensaje de confirmación después de completarlo.

Organización de los archivos JavaScript

Organizamos el código dentro de TP2/src/recursos/js/ y utilizamos módulos con `import` y `export` para compartir funciones entre archivos. Los scripts se cargan en el HTML mediante `type="module"`.

La organización quedó de la siguiente manera:

- `productos.js`: carga los productos desde el archivo JSON mediante `fetch()` y controla que la respuesta y los datos recibidos sean válidos.
- `funciones.js`: reúne las funciones para mostrar productos, aplicar filtros y leer, normalizar, guardar o vaciar el carrito.
- `index.js`: maneja el catálogo, la búsqueda, los filtros, los botones para agregar productos y el cuadro de confirmación.
- `carrito.js`: actualiza el contador, muestra el detalle del producto y administra el carrito, el cálculo del envío y la confirmación del pedido.
- `perfil.js`: recupera y guarda localmente los cambios del perfil.

Esta separación nos permite trabajar cada parte del sitio de forma más ordenada y reutilizar funciones.

Datos de los productos

Los datos de las prendas están en TP2/src/recursos/datos/productos.json. Cada producto tiene un identificador numérico, nombre, imagen, descripción, talles disponibles, colores, precio, categoría y género. Las categorías del desplegable se encuentran definidas en el HTML.

El archivo `productos.js` utiliza `fetch()` para obtener los datos. Si la respuesta no es correcta o el archivo no contiene una lista válida, se muestra un mensaje para informar que no se pudieron cargar los productos.

Renderizado de productos y filtros

En `funciones.js` usamos funciones que reciben los datos y el contenedor donde se debe mostrar la información.

La función `renderizarCatalogo(contenedor, productos)` recorre los productos con `forEach()`, arma las tarjetas mediante template literals y las incorpora a la página con `innerHTML`. Los enlaces y botones incluyen el atributo `data-id` para identificar el producto correspondiente. Los eventos de esos elementos se asignan desde `index.js`.

La función `renderizarDetalleProducto(contenedor, producto)` genera la ficha de la prenda seleccionada y completa las opciones de talle y color disponibles. También actualiza el título de la página y el nombre que aparece en la ruta de navegación.

Para identificar la prenda utilizamos `obtenerIdProductoSeleccionado()`, que primero busca el identificador en la URL y, si no lo encuentra, consulta el valor guardado en `localStorage`.

Los filtros por categoría y talle utilizan `filter()`. Para el talle también usamos `includes()` para comprobar si está disponible. La búsqueda compara el texto ingresado con el nombre, la descripción, la categoría y los colores del producto. Se puede combinar con los filtros de categoría y talle. Con “Ver todos” se restablecen las opciones y vuelve a mostrarse el catálogo completo.

Además, está definida la función `filtrarPorPrecio()`, aunque todavía no está conectada a un control de la página.

Interacción desde el catálogo

Al presionar “Añadir al carro”, se agrega una unidad del producto al carrito, se guarda el cambio y se actualiza el contador del encabezado.

Después se abre un cuadro de confirmación mediante `<dialog>` y `showModal()`. El botón “Seguir comprando” lo cierra a través de un formulario con `method="dialog"`.

Al seleccionar “Ver producto”, se guarda el identificador en `localStorage` y se navega al detalle mediante un enlace que también incluye ese identificador en la URL.

Funcionamiento del carrito

El carrito se guarda en `localStorage`, por lo que conserva su contenido al recargar la página o navegar entre las secciones del sitio. Para esto utilizamos `obtenerCarrito()`, `guardarCarrito()` y `vaciarCarrito()`, junto con `JSON.parse()` y `JSON.stringify()`.

El formato actual guarda el producto, el talle, el color y la cantidad. Al agregar varias veces la misma variante, se agrupan sus unidades. Las variantes de una misma prenda con distinto talle o color se muestran por separado. También se contempla el formato anterior, que guardaba solamente el identificador del producto.

Desde la ficha de una prenda se puede elegir la cantidad y agregarla al carrito. El talle y el color son obligatorios y la cantidad debe ser un número entero igual o mayor que uno.

Cada artículo presenta su imagen, nombre, variante, precio unitario y cantidad. El botón de eliminación quita la variante seleccionada y actualiza el listado, el contador y los importes.

Si el carrito está vacío, se muestra un mensaje con un enlace al catálogo y los importes quedan en cero. Cuando contiene productos, se calcula el subtotal y se suma un costo fijo de envío de $3.500. Para calcularlo se solicita un código postal de entre cuatro y ocho números; el costo informado es una estimación de referencia.

La sección “Te puede interesar” contiene tres prendas definidas en el HTML. Sus botones funcionan con JavaScript y permiten agregarlas directamente al carrito.

Antes de avanzar a la confirmación, se valida que haya productos en el carrito y que los datos requeridos estén completos. El resumen del pedido se guarda antes de navegar a la página de confirmación. Allí se muestran los productos, las variantes y los importes. Después de presentar un pedido válido, se vacía el carrito. La compra es una simulación y no se procesa ningún pago real.

Formularios y datos guardados

El formulario de contacto valida los campos requeridos y el formato del correo electrónico. El formulario del perfil valida los datos ingresados y guarda los cambios en `localStorage`, para recuperarlos al volver a esa página.

Los datos del carrito y del perfil quedan guardados en el navegador utilizado. No se sincronizan entre dispositivos.

Pruebas realizadas y limitaciones

Probamos el recorrido del catálogo, la búsqueda y los filtros, la selección de productos y variantes, el carrito, la persistencia al recargar, la eliminación, el cálculo estimado del envío, los formularios y la confirmación. También verificamos el carrito vacío y que una visita directa a la página de confirmación no confirme un pedido inexistente.

Los productos se cargan desde un archivo JSON local que simula una fuente de datos.